KPIT | APEX LAB

Breaking Bare-Metal Thinking

Why Super-Loops Fail and Why We Need Operating Systems

Disclaimer

This material has been developed exclusively for educational and academic purposes under the KPIT APEX Lab – Academic CSR Initiative. It is intended strictly for student learning, demonstrations, and classroom use.

Users are advised to verify and confirm the correctness of the contents before use.

It must not be copied, forwarded, shared, reproduced, or redistributed to any other individuals, groups, institutions, or platforms without prior written permission.

All examples, commands, diagrams, and demonstrations included here are meant solely to support conceptual understanding and learning.

Any use of these contents in commercial products, paid workshops, training programs, or proprietary documentation is strictly prohibited.

Power-On to Code Execution

Microcontroller with Bare-Metal Code

Reset
↓
Reset Handler
↓
Startup Code
↓
Calling main()

OS-Based Systems

Boot ROM
↓
Bootloader
↓
Kernel
↓
User Application

- On a microcontroller, there is no operating system.
- The CPU does exactly one thing: It jumps directly into your code.

Boot ROM = manufacturer-provided startup code stored in on-chip read-only memory.

Typical Boot Flow – BareMetal

- **Loads** initial **Stack Pointer**
- **Loads Program Counter**

Vector Table

[0] Initial Stack Pointer

[1] Reset Handler

[2] NMI Handler

[3] HardFault Handler

[...] other exceptions/handlers

- **Internal clock configuration** → PLL + crystal
- Configure Clock Tree and Peripheral Clock

Power-On Reset/ Reset Sources → Hardware Reset → Vector Table Fetch → Reset Handler → Memory Initialization → Calling main()

- **CPU registers** → initialized to **reset values**.
- **System Clock** → initialized to **internal RC oscillator** at a safe
- **SRAM contents are undefined** after reset
- **Vector table** resides at a 0x00000000 (often remapped)

- Copy .data: Values for **global/static** initialized variables live in Flash; copy them to RAM destination.
- Zero .bss: **Clear uninitialized globals/statics to zero** in RAM.
- Run system init hooks (SystemInit)
- Run C++ global constructors (__libc_init_array)

The Super-Loop Model

Consider sample example program

```
int main(void)
{
   init_uart();
   init_sensor();
   init_led();

   while (1)
   {
      read_sensor();
      process_data();
      send_uart();
      toggle_led();
      delay_ms(100);
   }
}
```

If each function takes 10 ms, what is the loop frequency?

Key Observations

How many programs are running?	Who decides what runs next?	What decides timing?
One	You	Delays and timers

This model works well for tiny systems but collapses as features grow.

The Blocking Problem

- Consider sample example program

```
int main(void)
{
   init_uart();
   init_sensor();
   init_led();

   while (1)
   {
      read_sensor();
      delay_ms(50)
      send_uart();
      delay_ms(300)
      toggle_led();
      delay_ms(10);
   }
}
```

"When UART blocks for 300 ms, what happens to sensor sampling cadence?"

"If you need sensor sampling every 50 ms, can this meet the deadline?"

- Delayed sampling violates timing guarantees—late samples are wrong for control systems.
- You cannot guarantee periodic sampling with blocking calls in a serialized loop.
- Reordering does not fix the fundamental serialization.

Blocking in one feature propagates to all features in a super-loop.

Limitations of Super-Loop Firmware

Scalability

What happens when you add:

- Ethernet?
- USB?
- Logging?
- Display?

→ The loop grows… and grows… and breaks.

Adding functions increases latency unpredictably—each function's run-time adds up in the loop.

Maintainability

"You want to add a logging feature. You add calls in the loop and a few global flags.
Two months later, a teammate removes an obsolete feature. Now logging breaks."

- One change affects everything, Hard to add/remove features
- Tight coupling in shared globals and timing assumptions cause hidden dependencies.

Reliability

A single bug (e.g., NULL pointer in display code) crashes the entire controller. Is there any containment?
No recovery mechanism

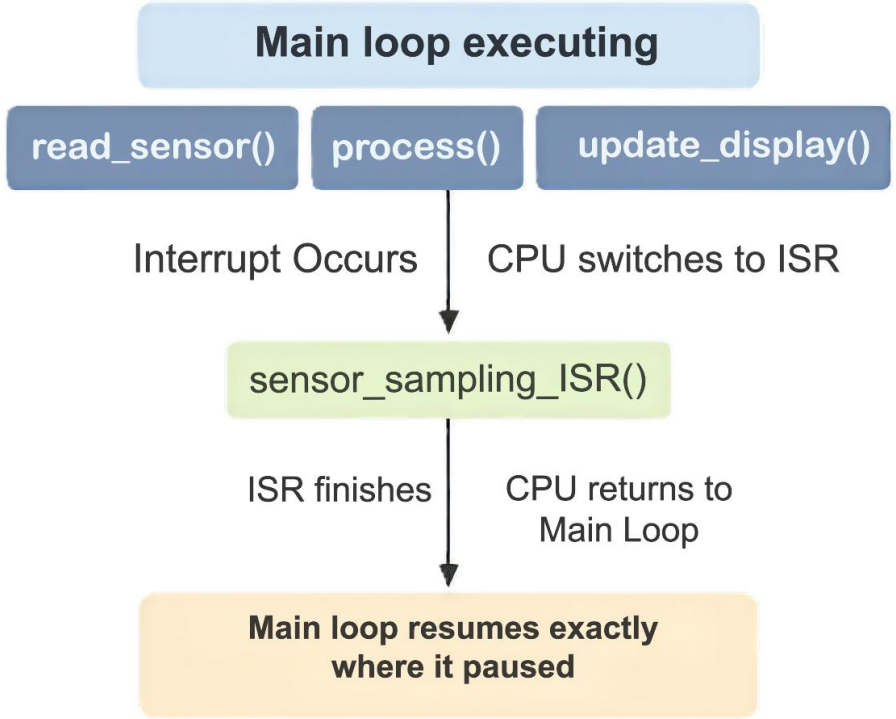
- Watchdogs only reset—they do not contain faults or preserve state.
- Defensive coding helps but cannot isolate failures.
- Bare-metal has no memory protection—a write out of bounds corrupts unrelated features.

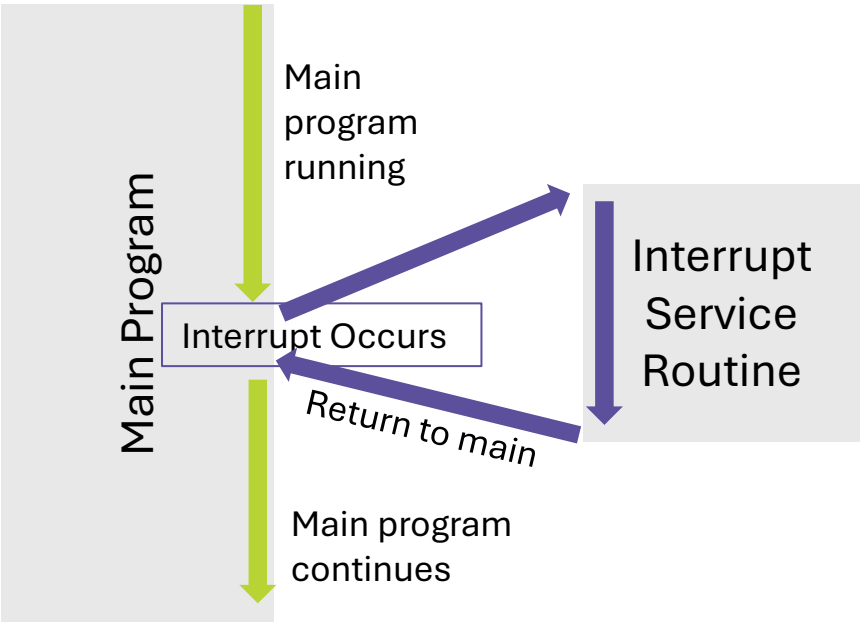
Feature Isolation

Sensor code and networking code share buffers and timing. Networking stalls; sensor data stops being sampled on time.

- In bare-metal systems, every feature trusts every other feature.
- Double buffers help data flow but do not isolate faults or guarantee scheduling.
- Priority in bare metal is manual and fragile; if high-priority work starves others, they miss deadlines.

Interrupt Driven Design





- Interrupts do not create parallelism.
- Interrupts **only pause** the main loop — they do not replace it.
- Interrupt based design will not solve the super loop concerns.

Interrupts help responsiveness, not concurrency.

Interrupts help responsiveness but do not provide isolation or scheduling guarantees.

Scenario - Situation (1/4)

Develop a solution for designing Smart Industrial Controller

Features:

- Sensor sampling: Read temperature every 50 ms; processing must finish within 10 ms of each sample.
- Display update: Refresh UI at 10 Hz (every 100 ms).
- Alarm monitoring: Check thresholds continuously; alarm must be raised within 20 ms.
- Networking: Send telemetry packets every 500 ms; network stack may block up to 200 ms per send.
- Logging: Write to flash every 1 minute; flash erase can block for 300 ms.

Constraints:

- Single super-loop only, no OS (bare-metal mindset).
- You may use interrupts for sensor sampling.
- No multitasking, no threads/processes.
- Assume blocking I/O for networking and flash.

Tasks:

- Decide order and delays.
- If using an ISR for sensor sampling, specify ISR duration and main-loop interactions.
- Identify worst-case latencies for each feature.
- Predict failures under load (e.g., slow network send + flash logging overlap).

Scenario - Situation (2/4)

Requirements:

Feature	Period	Deadline	Worst-Case Blocking
Sensor sampling	50 ms	10 ms processing	ISR + main loop delay
Display	100 ms	soft deadline	3–5 ms typical
Alarm	continuous	20 ms	Depends on network/logging blocking
Network	500 ms	no hard deadline	200 ms blocking
Flash logging	1 minute	none	300 ms blocking

Timeline

Sensor sampling	|-----50 ms -----|-----50 ms -----|-----50 ms -----|-----50 ms -----|
Display	|----------100 ms ----------|----------100 ms ----------|
Network	|----------500 ms ----------|
Flash logging	|----------1 min----------|

Scenario – Typical Solution (3/4)

```
while(1) {
read_sensor(); // 10 ms
process_data(); // 5 ms
display_update(); // 3 ms
alarm_check(); // 2 ms
network_send(); // 200 ms (blocking worst case)
log_to_flash_if_due(); // 300 ms (blocking worst case)
delay_ms(10); // try to pace loop
}
```

"We'll ensure sensor is first so it's timely."

"We'll put logging at the end so it doesn't interfere."

"We'll run alarm checks very frequently."

"Which deadlines were violated and why?"

"Could you guarantee alarm response within 20 ms during logging?"

"Could you guarantee sensor processing within 10 ms when network blocks?"

- deadlines were violated due to blocking and serialized work.
- Alarm response cannot be guaranteed in presence of 200–300 ms blocking operations.
- Sensor processing cannot be guaranteed because even if sampling is in ISR, post-processing and actuation depended on main loop.

Scenario - The Solution (4/4)

Scheduling	• avoids blocking delays
Isolation	• memory protection between components
Abstractions	• drivers, file systems, sockets

Bare-metal Problem → OS Solution

Bare-metal Problem	OS Solution
One big loop → serialization	Preemptive scheduling
Blocking network/logging	Threads + asynchronous I/O
No isolation	Memory protection
Fault in one function kills system	Process isolation / watchdog restart
Feature interactions	Drivers + queues + buffers
Manual timing	OS tick / scheduler

- An OS doesn't eliminate blocking — it isolates it so the rest of the system continues running.
- As features scale, concurrency, isolation, and scheduling become mandatory — a super-loop can no longer meet system requirements.

Operating System Benifits

A complex embedded product requires an Operating System to provide:

Scheduling	Who runs when	preemption, priorities
Isolation	One bug ≠ total failure	process boundaries, memory protection
Abstractions	Uniform hardware access	non-blocking, async, buffered
Reliability	Predictable timing under load and/or real-time extensions	Controlled failures

Summary

Super-loop = simple but fragile

Interrupts ≠ concurrency

Blocking destroys timing

Real systems need a scheduler + isolation